The Computing Ideas Behind the Volatility Surface

A plain-language guide to the scientific computing used in the MATLAB project

Kevin Williams | October 23, 2026

The one-minute version

Stock options are quoted as *prices*, but what people actually care about is *volatility*: how bumpy the market expects a stock to be. Nobody quotes volatility directly. This project takes real option prices and works backward to the volatility hiding inside each one, then paints all those volatilities onto a single surface. Doing that reliably takes a handful of ideas from scientific computing. None needs advanced math to understand; each is a sensible way of dealing with a messy, imperfect world.

The big picture in four steps

1. Write down a formula linking price and volatility. **2.** Run the formula backward, one option at a time. **3.** Join the results into a smooth picture. **4.** Prove the answers can be trusted.

1 Principle 1: a model is a simplified story

The formula used here, Black-Scholes, tells a simple story: a stock drifts and wiggles randomly, and an option's price is what that wiggling is worth today. The story is wrong in details (real prices jump, and volatility itself changes), but it is useful because it has exactly one number the market must supply: volatility. Setting the formula's price equal to the market's price and solving for that number gives *implied volatility*. Real markets disagree with the simple story in a very informative way: different strikes give different answers, and that pattern is the smile and surface the project draws.

2 Principle 2: let the computer do the algebra, once

Working backward needs a rate of change: how much does the price move if volatility moves a little? Traders call it "vega." Rather than copying a formula from a textbook, the project asks MATLAB's *symbolic* engine to derive it from the pricing formula. The same engine then checks two facts the formula must obey (the equation that all option prices must satisfy, and a relationship tying call prices to put prices). Symbolic computing is like writing a recipe once, exactly, and then converting it into a fast kitchen routine that cooks thousands of dishes. Exact algebra first, fast arithmetic second.

3 Principle 3: solve by guess-and-correct

There is no tidy formula that turns a price into a volatility, so the project plays a "hotter or colder" game. Guess a volatility, compute the price it would give, see how far off that is, and use the rate of change to make a much better guess. This is *Newton's method*, and it usually lands on the answer in a handful of steps.

Round	Guess for volatility	Price error
1	30% (a starting guess)	large
2	a much closer guess	small
3	very close	tiny
4	answer to 8 decimals	essentially zero

(The pattern is typical, not a record of one run.)

4 Principle 4: always keep a safety net

Guess-and-correct can occasionally jump somewhere silly, much as a hiker following the steepest slope in fog can step off the path. So the solver also keeps a *bracket*: a lower and an upper bound that the true answer must sit between. If a guess lands outside the bracket, the solver instead takes the safe route of splitting the bracket in half (*bisection*), which is slower but cannot fail. The result is the speed of Newton with the reliability of bisection. If a price is impossible, for example a call worth more than the stock itself, no volatility can produce it and the solver reports "not a number" instead of inventing an answer.

5 Principle 5: know which questions have fuzzy answers

Some questions are hard to answer because the data hardly responds to the thing being asked. A deeply in-the-money option is mostly just "stock minus strike"; its price barely changes when volatility changes, so a one-cent pricing error can swing the implied volatility by many points. The rule of thumb is simple:

$$\text{volatility error} \approx \frac{\text{price error}}{\text{vega}}.$$

This is called *conditioning*. The project handles it by using only *out-of-the-money* options, whose prices are all "volatility value" and therefore informative. The same reasoning explains why some test cases are skipped: where nothing can be learned from the price, a test that demands an answer would be unfair.

6 Principle 6: use relationships to remove unknowns

To compare options on a fair footing, the project needs the stock's *forward price*: where the market expects it to be on the expiry date, after interest and dividends. Instead of guessing it, the project reads it off the market using put-call parity. A call and a put with the same strike and expiry, bought and sold together, behave like a fixed bet on the forward price, so their price difference reveals it. Using several nearby strikes and taking the median (the middle value) keeps one bad quote from spoiling the estimate.

7 Principle 7: connect the dots without inventing data

After solving, the project has volatility at scattered points: a strike here, an expiry there. To draw a surface it fills in the gaps with *interpolation*, which is like laying a mesh of flat triangular tiles across the dots. Three details matter.

- **Even out the axes first.** Strike distances range over about ± 0.25 while time ranges over only about 0.02 to 0.2 years. Triangles built on such uneven axes come out as skinny slivers, so both axes are rescaled to the same 0–1 range.
- **Interpolate the right quantity.** Instead of volatility, the code interpolates *total variance* (volatility squared times time). It grows steadily with time, so stitching across expiries behaves sensibly.
- **Leave blanks blank.** Outside the region covered by real quotes the surface is left empty. Extrapolating beyond the data would be a confident-looking guess.

8 Principle 8: rounding, missing values and silent traps

Computers store numbers with limited precision, and small details can quietly wreck a result.

- The OCaml engine prints volatilities to six decimals, so any two programs can differ by up to 5×10^{-7} without either being wrong. That is rounding, not error.
- A real bug found during this project: MATLAB's `max(NaN, 0)` returns 0, so a blank region of the surface was quietly turned into "zero volatility." The fix was to clamp only negative numbers and leave missing values missing, and a test now guards against it.
- Real data is dirty. The exporter drops quotes with no bid, wide spreads or no open interest before any math begins. Good math cannot rescue bad inputs.

9 Principle 9: trust is earned by checking

A result that merely *looks* right is not evidence. The project uses four independent kinds of proof:

1. **Known answers.** Compare against a worked example whose answer is fixed by the formula.
2. **Round trips.** Turn a volatility into a price and back; you should recover what you started with.
3. **Planted truth.** Build fake market data from a smile you chose, then check the tool recovers it, including at points it never saw.
4. **Independent implementations.** Solve the same real data with a different program (the OCaml engine, and separately SciPy in Python). Agreement between unrelated programs is strong evidence.

The companion *Verification Process* document walks through all four with numbers.

10 Glossary

Option	A contract giving the right, not the obligation, to buy (call) or sell (put) a stock at a set price (the strike) by a set date (the expiry)
Volatility	How widely a stock's price tends to swing, as a yearly percentage
Implied volatility	The volatility that makes the formula match the market price
Smile / skew	How implied volatility changes with strike
Term structure	How implied volatility changes with time to expiry
Forward price	Where the market expects the stock to be at expiry
Vega	How much an option's price moves per unit change in volatility
Interpolation	Estimating values between known data points
NaN	"Not a number": the computer's marker for a missing or impossible value